Gradient Descent and Optimization Techniques

ML-101 Lecture 2

Rose Alsalman

Massachusetts Institute of Technology

January 2025

Table of Contents

Table of Contents

Linear Regression: Analytical Solution

Linear regression is a method to model the relationship between a dependent variable y and one or more independent variables X.
Objective: Minimize the Mean Squared Error (MSE) loss:

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(\theta^T x^{(i)} - y^{(i)})^2.$$

Analytical Solution: The parameters θ can be computed directly:

$$\theta = (X^T X)^{-1} X^T Y.$$

Advantages of Analytical Solutions:

- Provides exact results.
- Requires no iterative computation.

However, analytical solutions are not always feasible or efficient.

Challenges with Analytical Solutions

Computational Complexity:

- Inverting large matrices like $(X^T X)^{-1}$ is computationally expensive for high-dimensional data.
- Example: A dataset with millions of features would require immense memory and time to compute the inverse.

Large Datasets:

- For massive datasets, computing gradients over the entire dataset at once is infeasible.

Why Gradient Descent?

- Gradient descent provides a numerical approach that can handle these challenges efficiently.
- It works for both simple and complex objective functions, including non-linear and high-dimensional cases.

Why Do We Care About Gradient Descent?

Key Benefits:

- Scalability: Works efficiently for large datasets by using mini-batches or stochastic updates.
- Flexibility: Can optimize any differentiable function, including non-linear and non-convex functions.
- Universality: Applicable to a wide range of problems, from regression to deep learning.

Intuition Behind Gradient Descent:

- Imagine a surface representing the cost function $J(\theta)$.

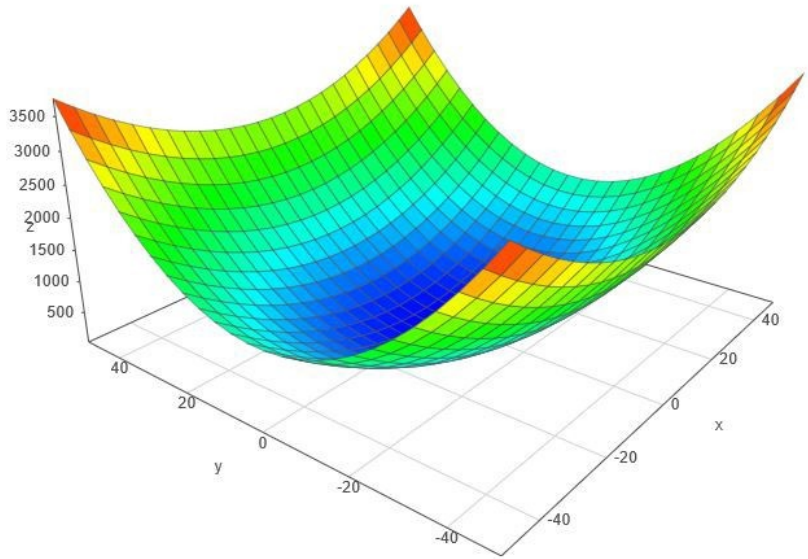

Figure:

Intuition Behind Gradient Descent:

- Imagine a surface representing the cost function $J(\theta)$.
- Gradient descent starts at an arbitrary point on this surface and iteratively moves in the direction of steepest descent until it reaches a minimum.

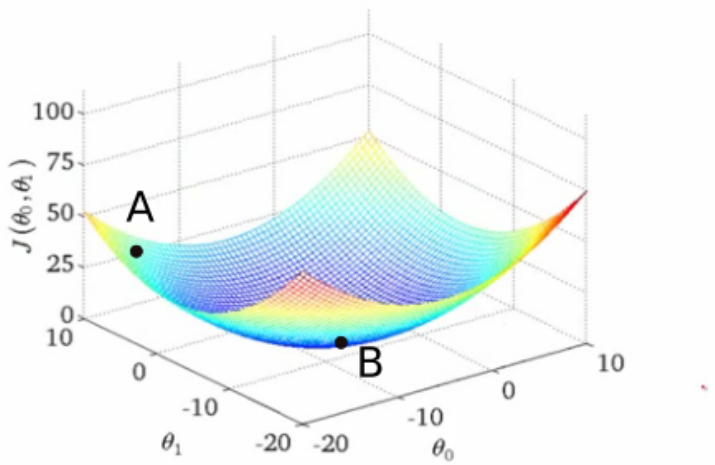

Figure:

Table of Contents

Critical points for a multi-variable function

For a function $f(x, y)$, to find the minimum, we set the gradient vector equal to zero or find values where it is undefined:

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = (0, 0).$$

This means that at critical points:

$$\frac{\partial f}{\partial x} = 0 \quad \text{and} \quad \frac{\partial f}{\partial y} = 0.$$

Because critical points are candidates for local minima. To classify these points, we use the **Second Partial Derivative Test**.

Classifying Critical Points

$$f_{xx} = \frac{\partial^2 f}{\partial x^2}, \quad f_{yy} = \frac{\partial^2 f}{\partial y^2}, \quad f_{xy} = \frac{\partial^2 f}{\partial x \partial y}.$$

At a critical point (x_0, y_0), compute the determinant of the Hessian matrix:

$$D = f_{xx}(x_0, y_0) f_{yy}(x_0, y_0) - [f_{xy}(x_0, y_0)]^2.$$

Classification:

1. If $D > 0$ and $f_{xx}(x_0, y_0) > 0$, then $f(x, y)$ has a **local minimum** at (x_0, y_0).
2. If $D > 0$ and $f_{xx}(x_0, y_0) < 0$, then $f(x, y)$ has a **local maximum** at (x_0, y_0).
3. If $D < 0$, then (x_0, y_0) is a **saddle point**.
4. If $D = 0$, the test is inconclusive.

What is Gradient Descent?

Gradient descent is an optimization algorithm used to minimize functions by iteratively moving in the direction of steepest descent as defined by the negative gradient.

- Objective: Minimize $J(\theta)$, where J is the cost function.
- Update rule: $\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$.
- $\eta > 0$: Learning rate or step size.
- Works well for convex functions; may converge to local minima for non-convex functions.

Table of Contents

Gradient Descent Algorithm (1D)

For a one-dimensional function $f(x)$:

Algorithm Steps

1. Initialize x_0. 2. Repeat until convergence:

$$x_{t+1} = x_t - \eta f'(x_t)$$

3. Stop when $|f(x_{t+1}) - f(x_t)| < \epsilon$.

Common stopping criteria:

- Small change in $f(x)$.
- Small gradient magnitude ($|f'(x)| < \epsilon$).
- Fixed number of iterations.

Gradient Descent Algorithm (1D) Sudo Code

Input: $\Theta_{\text{init}}, \eta, f, f', \epsilon$
Output: $\Theta^{(t)}$
$\Theta^{(0)} = \Theta_{\text{init}}$;
$t = 0$;
repeat
  $t = t + 1$;
  $\Theta^{(t)} = \Theta^{(t-1)} - \eta f'(\Theta^{(t-1)})$;
until $|f(\Theta^{(t)}) - f(\Theta^{(t-1)})| < \epsilon$;
return $\Theta^{(t)}$;

Algorithm 1: 1D-GRADIENT-DESCENT

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x$

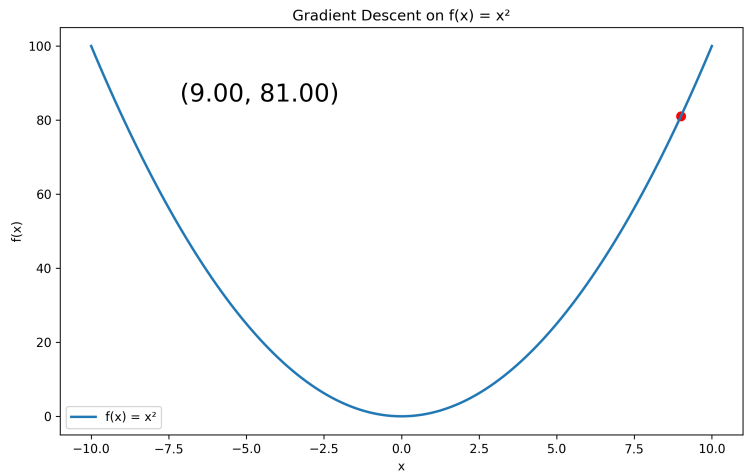

Figure: Initial point: x_0

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x \quad x_1 = x_0 - \eta \nabla f(x_0) = 9 - 0.8(2 \cdot 9) = -5.4$

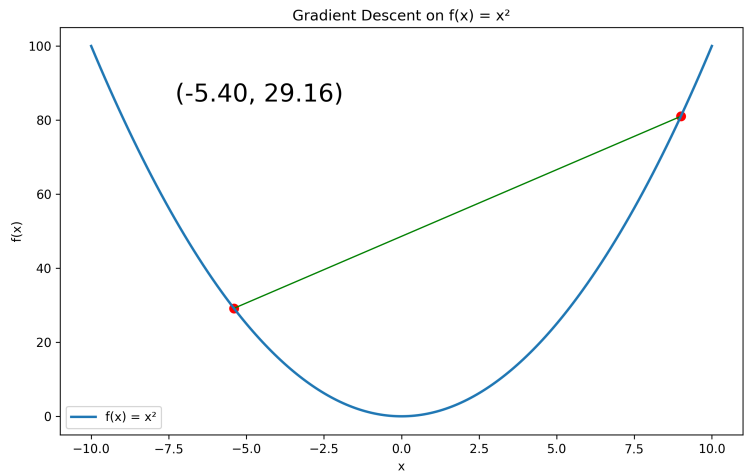

Figure: Updated to x_1

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x \quad x_2 = x_1 - \eta \nabla f(x_1) = 5.4 - 0.8(2 \cdot -5.4) = 3.24$

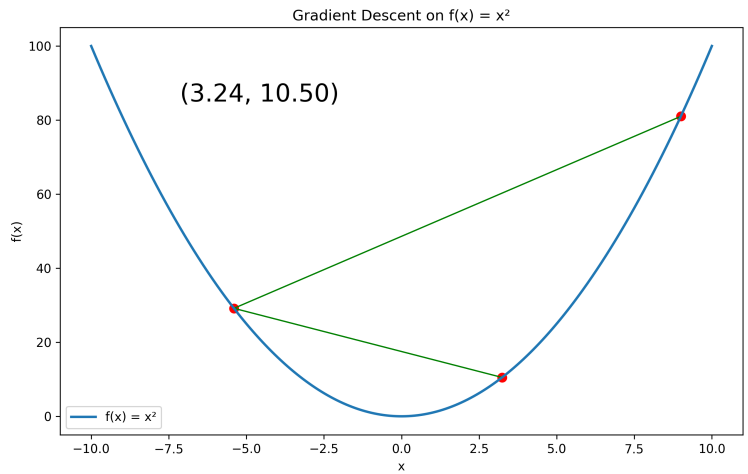

Figure: Updated to x_2

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x \quad x_3 = x_2 - \eta \nabla f(x_2) = 3.24 - 0.8(2 \cdot 3.24) = -1.944$

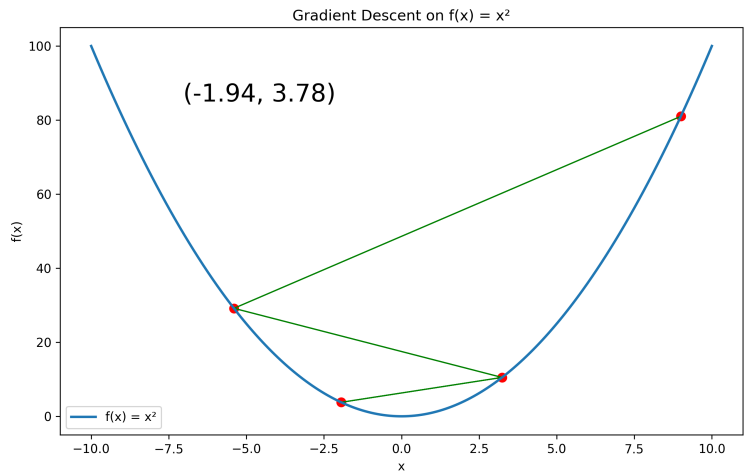

Figure: Updated to x_3

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x \qquad x_4 = x_3 - \eta \nabla f(x_3) = 1.944 - 0.8(2 \cdot -1.944) = 1.1664$

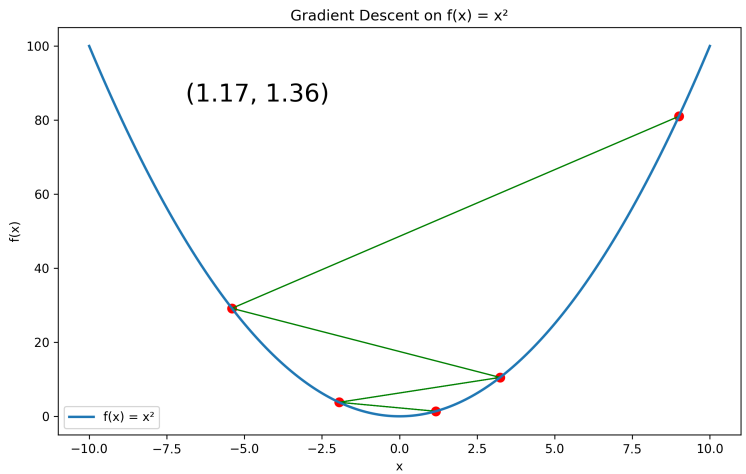

Figure: Updated to x_4

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x$

  $x_5 = x_4 - \eta \nabla f(x_4) = 1.1664 - 0.8(2 \cdot 1.1664) = -0.69984$

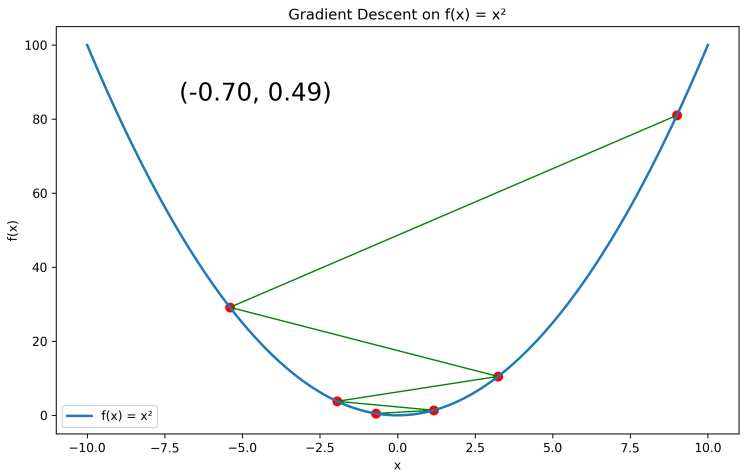

Figure: Updated to x_5

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x$

  $x_6 = x_5 - \eta \nabla f(x_5) = 0.69984 - 0.8(2 \cdot -0.69984) = 0.419904$

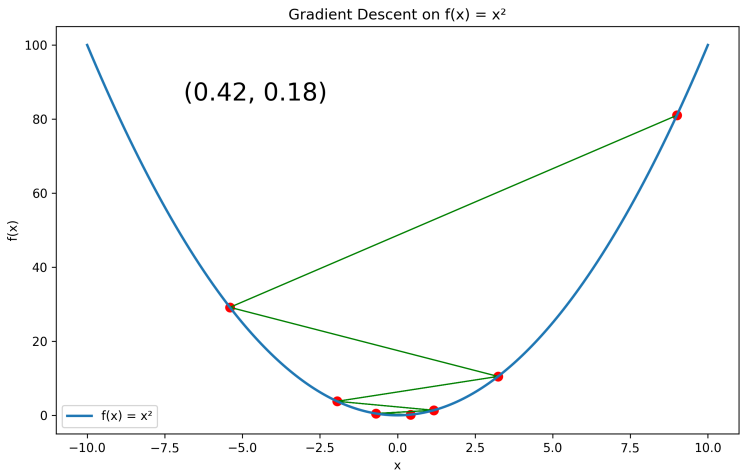

Figure: Updated to x_6

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x$

  $x_7 = x_6 - \eta \nabla f(x_6) = 0.419904 - 0.8(2 \cdot 0.419904) = -0.2519424$

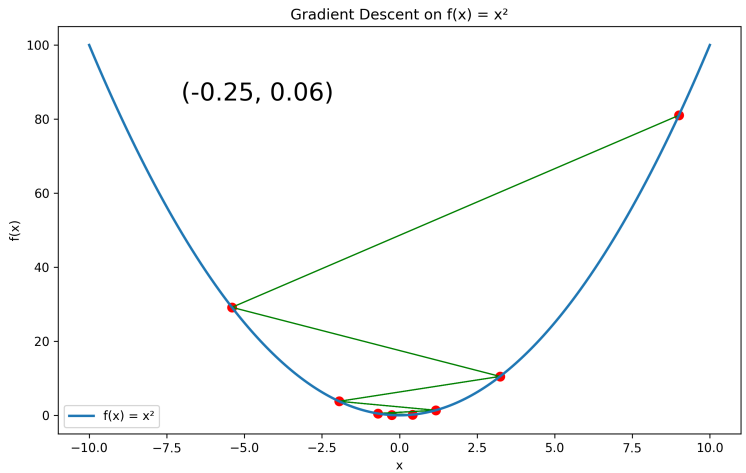

Figure: Updated to x_7

1D Gradient Descent Example

Let's consider the function $f(x) = x^2$ and take an initial guess:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.8$
- $\nabla f(x) = 2x$

  $x_8 = x_7 - \eta \nabla f(x_7) = 0.2519424 - 0.8(2 \cdot -0.2519424) = 0.15116544$

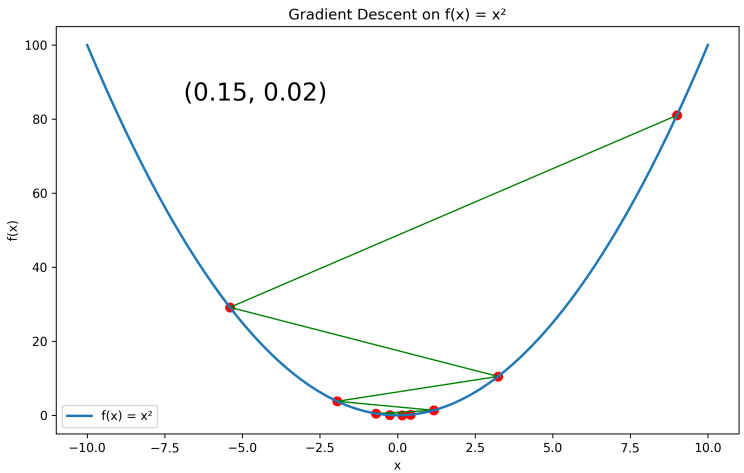

Figure: Updated to x_8

Gradient Descent in Multiple Dimensions

For $f : \mathbb{R}^m \to \mathbb{R}$, update rule becomes:

$$x_{t+1} = x_t - \eta \nabla f(x_t),$$

where $\nabla f(x_t) = [\partial f/\partial x_1, ..., \partial f/\partial x_m]^T$.
Key considerations:

- Step size (η) affects convergence rate.
- Convexity ensures global minimum; non-convexity may lead to local minima.
- Smoothness assumptions improve convergence guarantees.

Gradient Descent for Linear Regression: Gradient and Updates

- The gradient of the OLS objective with respect to parameters θ is:

$$\nabla_\theta J = \frac{2}{n} X^T (X\theta - Y)$$

where:

  - X: Matrix of features ($n \times d$).
  - Y: Vector of actual outputs ($n \times 1$).

- Using gradient descent, we iteratively update the parameters:

$$\theta^{(t)} = \theta^{(t-1)} - \eta \frac{2}{n} \sum_{i=1}^{n} (\theta^{(t-1)^T} x^{(i)} - y^{(i)}) x^{(i)}$$

where:

  - t: Iteration number.
  - $\eta > 0$: Learning rate controlling step size.
  - Updates stop when convergence is achieved (e.g., small gradient norm).

Ridge Regression: Objective and Gradients

- Ridge regression adds a regularization term to the ordinary least squares (OLS) objective:

$$J_{\text{ridge}}(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^{n} \left(\theta^T x^{(i)} + \theta_0 - y^{(i)} \right)^2 + \lambda \|\theta\|^2$$

- Gradients are computed separately for θ and θ_0:

$$\nabla_\theta J_{\text{ridge}}(\theta, \theta_0) = \frac{2}{n} \sum_{i=1}^{n} (\theta^T x^{(i)} + \theta_0 - y^{(i)}) x^{(i)} + 2\lambda\theta$$

and:

$$\frac{\partial J_{\text{ridge}}(\theta, \theta_0)}{\partial \theta_0} = \frac{2}{n} \sum_{i=1}^{n} (\theta^T x^{(i)} + \theta_0 - y^{(i)})$$

where:

- $\nabla_\theta J_{\text{ridge}}(\theta, \theta_0)$: Gradient with respect to θ, shape $d \times 1$.
- $\partial J_{\text{ridge}} / \partial \theta_0$: Gradient with respect to θ_0, a scalar.
- Regularization affects only $\nabla_\theta J_{\text{ridge}}(\theta, \theta_0)$, not $\partial J_{\text{ridge}} / \partial \theta_0$.

Ridge Regression: Gradient Descent Algorithm

Input: $\theta_{\text{init}}, \theta_{0,\text{init}}, \eta, \epsilon$

Output: $\theta^{(t)}, \theta_0^{(t)}$

Initialize: $\theta^{(0)} = \theta_{\text{init}}, \theta_0^{(0)} = \theta_{0,\text{init}}, t = 0$

repeat

$t = t + 1$;

Update $\theta^{(t)}$: $\theta^{(t)} =$

$$\theta^{(t-1)} - \eta \left(\frac{1}{n} \sum_{i=1}^{n} (\theta^{(t-1)^T} x^{(i)} + \theta_0^{(t-1)} - y^{(i)}) x^{(i)} + \lambda \theta^{(t-1)} \right)$$

Update $\theta_0^{(t)}$:

$$\theta_0^{(t)} = \theta_0^{(t-1)} - \eta \left(\frac{1}{n} \sum_{i=1}^{n} (\theta^{(t-1)^T} x^{(i)} + \theta_0^{(t-1)} - y^{(i)}) \right)$$

until $J_{ridge}(\theta^{(t)}, \theta_0^{(t)}) - J_{ridge}(\theta^{(t-1)}, \theta_0^{(t-1)}) < \epsilon$;

return $(\theta^{(t)}, \theta_0^{(t)})$

Algorithm 2: RR-GRADIENT-DESCENT

Table of Contents

Choosing the Learning Rate (η)

Impact of learning rate:

- Too small: Slow convergence.
- Too large: May overshoot or diverge.
- "Just right": Balances convergence speed with stability.

1D Gradient Descent Example

Let's Again consider the function $f(x) = x^2$ and take an initial guess but use a smaller learning rate:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.3$
- $\nabla f(x) = 2x$

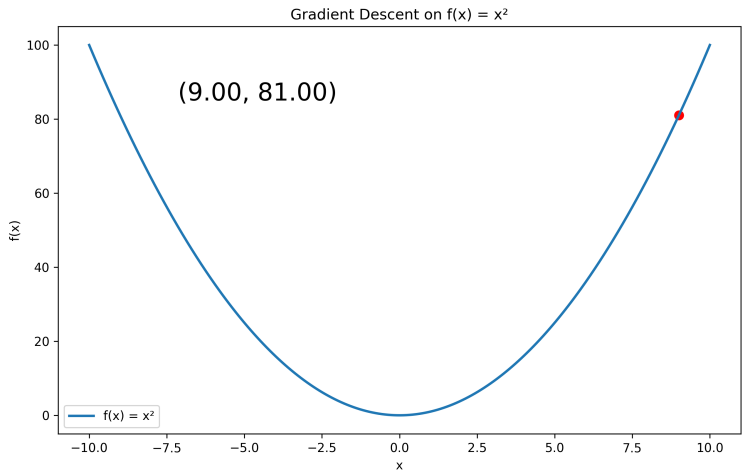

Figure: Initial point: x_0

1D Gradient Descent Example

Let's Again consider the function $f(x) = x^2$ and take an initial guess but use a smaller learning rate:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.3$
- $\nabla f(x) = 2x \quad x_1 = x_0 - \eta \nabla f(x_0)$

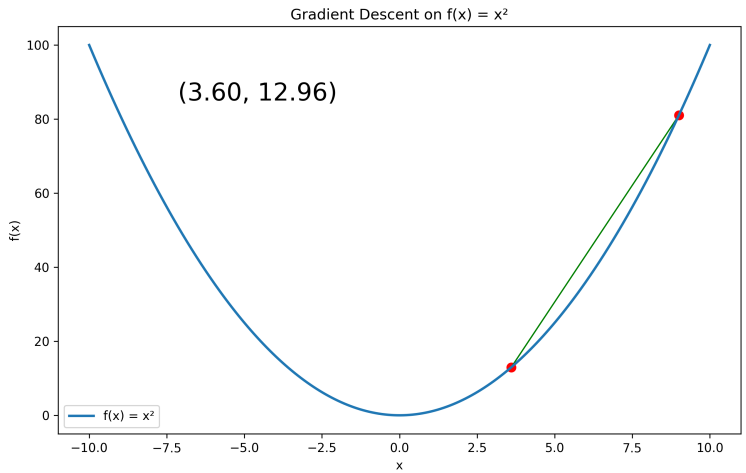

Figure: Updated to x_1

1D Gradient Descent Example

Let's Again consider the function $f(x) = x^2$ and take an initial guess but use a smaller learning rate:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.3$
- $\nabla f(x) = 2x \quad x_2 = x_1 - \eta \nabla f(x_1)$

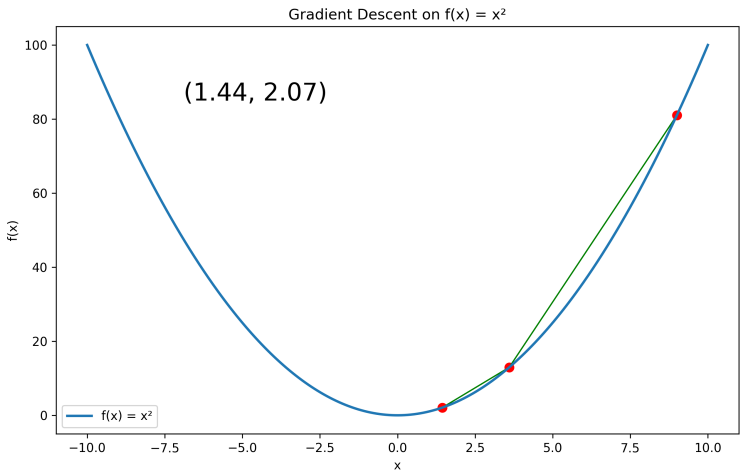

Figure: Updated to x_2

1D Gradient Descent Example

Let's Again consider the function $f(x) = x^2$ and take an initial guess but use a smaller learning rate:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.3$
- $\nabla f(x) = 2x \quad x_3 = x_2 - \eta \nabla f(x_2)$

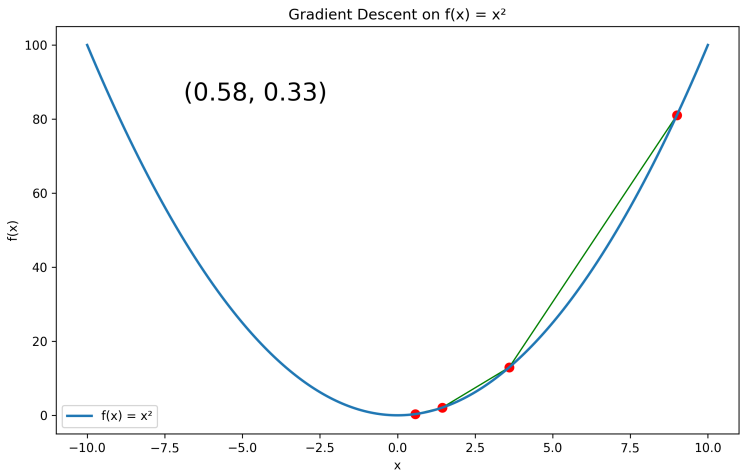

Figure: Updated to x_3

1D Gradient Descent Example

Let's Again consider the function $f(x) = x^2$ and take an initial guess but use a smaller learning rate:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.3$
- $\nabla f(x) = 2x \qquad x_4 = x_3 - \eta \nabla f(x_3) 4$

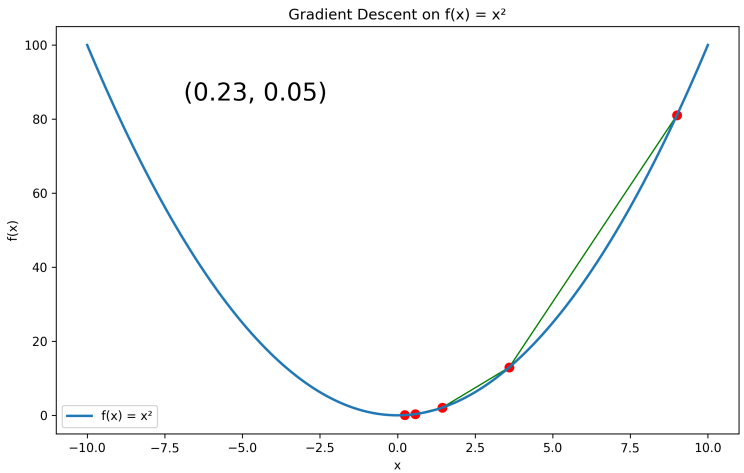

Figure: Updated to x_4

1D Gradient Descent Example

Let's Again consider the function $f(x) = x^2$ and take an initial guess but use a smaller learning rate:

- $x_0 = 9$ and $f(x) = 81$
- Learning rate $\eta = 0.3$
- $\nabla f(x) = 2x \qquad x_5 = x_4 - \eta \nabla f(x_4)$

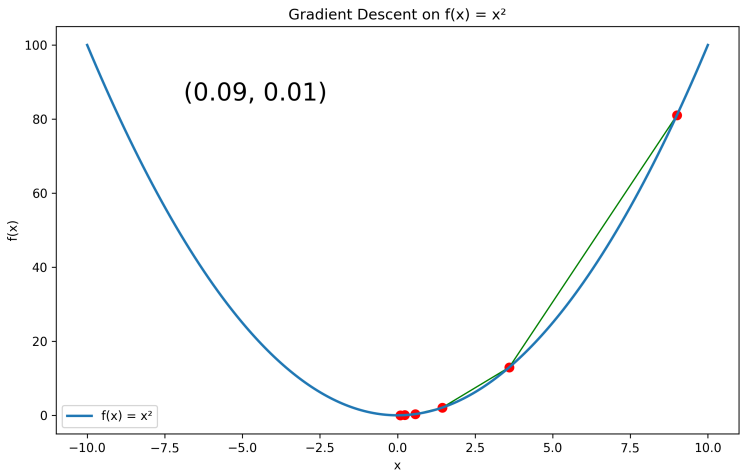

Figure: Updated to x_5

Stochastic Gradient Descent (SGD): Algorithm

SGD is particularly useful for large datasets where computing gradients over the entire dataset is computationally expensive.

Algorithm: Stochastic Gradient Descent

1. Initialize parameters θ_0. 2. For $t = 1$ to T:
   - Randomly select a data point i from the dataset.
   - Update:
   $$\theta_t = \theta_{t-1} - \eta_t \nabla f_i(\theta_{t-1}),$$
   where $f_i(\theta)$ is the loss for data point i.
3. Return θ_T.

Key considerations:

- Learning rate η_t often decreases with time (e.g., $\eta_t = 1/t$).
- Stopping criteria: Fixed number of iterations or convergence of loss.

Stochastic Gradient Descent (SGD): Pseudocode

Input: $\Theta_{\text{init}}, \eta, f, \nabla_\Theta f_1, \ldots, \nabla_\Theta f_n, T$
Output: $\Theta^{(T)}$
Initialize $\Theta^{(0)} = \Theta_{\text{init}}$;
for $t = 1$ *to* T **do**
    Randomly select $i \in \{1, 2, ..., n\}$;
    Update parameters:

$$\Theta^{(t)} = \Theta^{(t-1)} - \eta^{(t)} \nabla_\Theta f_i(\Theta^{(t-1)})$$

end
return $\Theta^{(T)}$

Algorithm 3: STOCHASTIC-GRADIENT-DESCENT

Advantages and Challenges of SGD

Advantages:

- Faster updates for large datasets.
- Can escape shallow local minima in non-convex functions.
- May generalize better by avoiding overfitting.

Challenges:

- High variance in updates due to random sampling.
- Requires careful tuning of learning rate schedule.
- Convergence can be noisy and slow.

Table of Contents

Gradient Descent vs. Stochastic Gradient Descent (SGD)

Aspect	Gradient Descent	Stochastic Gradient Descent (SGD)
Update Frequency	Once per epoch (entire dataset).	After each data point or mini-batch.
Computation Cost	High for large datasets.	Low per iteration.
Convergence	Smooth but slower.	Noisy but faster.
Memory Usage	Requires full dataset in memory.	Processes one or few samples at a time.

Use **SGD** when:

- Dataset is very large.
- Faster updates are required, even if noisy.
- Generalization is more important than precise convergence.

Table of Contents

Conclusion and Takeaways

Gradient descent is a foundational optimization algorithm with wide applications in machine learning.

Key Points:

- Works by iteratively updating parameters in the direction of negative gradient.
- Variants like SGD improve efficiency for large datasets.
- Learning rate and regularization are critical for performance.

Thank you! Questions?